

1 Algoritmos

Podemos considerar um algoritmo como uma sequência de finita de instruções que resolve um problema computacional ou ferramenta para resolver um **problema computacional** bem especificado. O enunciado do problema especifica em termos gerais a relação desejada entre entrada e saída. O algoritmo descreve um procedimento computacional específico para se conseguir essa relação entre entrada e saída.

1.1 Problemas e suas instâncias

Todo problema computacional é uma coleção de casos particulares que chamaremos instâncias (= instances). Uma instância é especificada quando atribuímos valores aos parâmetros do problema. Em outras palavras, uma instância é especificada por um particular conjunto de dados do problema.

1.2 Tamanho de uma instância

O tamanho (= size) de uma instância de um problema é a quantidade de dados necessária para descrever a instância (ou seja, o espaço de memória necessário para especificar os valores dos parâmetros que definem a instância). Em geral, o tamanho de uma instância é descrito por um único número natural, mas às vezes é mais conveniente usar dois ou mais números.

1.3 Exemplo

- **Problema:** Multiplicação de números naturais: Dados números naturais u e v , encontrar a expansão decimal do produto $u * v$.
- **Tamanho da instância:** No problema da multiplicação de números naturais, por exemplo, parece razoável dizer que toda instância têm tamanho 2 (pois toda instância é especificada por dois números).
- **Solução:** É o conjunto unitário contendo o resultado de $u * v$

2 Análise assintótica de funções

Ao estudar expressões como $n + 10$ ou $n^2 + 1$, é comum que as pessoas inicialmente se concentrem nos valores específicos de n , como 1, 2, 3, e assim por diante. Por exemplo, para $n + 10$, quando $n = 1$, o resultado é 11; quando $n = 2$, o resultado é 12, e assim por diante. Essa abordagem é útil para entender o comportamento das expressões em casos particulares e para cálculos práticos em situações específicas.

No entanto, quando estamos analisando algoritmos e buscamos entender seu desempenho à medida que o tamanho do problema (representado por n) cresce, olhar apenas para valores pequenos de n pode não nos fornecer uma visão completa. Em vez disso, nos concentramos nos valores enormes de n .

A matemática assintótica nos ajuda a entender como o desempenho de um algoritmo (ou o crescimento de uma função) se comporta à medida que o tamanho do problema cresce indefinidamente. Em vez de nos preocuparmos com detalhes específicos para valores particulares de n , estamos interessados principalmente em como o desempenho ou o crescimento se comporta quando n se aproxima do infinito.

2.1 Notação Θ

Para uma dada função $g(n)$, denotamos por $\Theta(g(n))$ o conjunto de funções

$\Theta(g(n)) = f(n)$: existem constantes positivas c_1, c_2 e n_0 tais que

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ para todo } n \geq n_0$$

Ou seja, uma função $f(n)$ é igual a $\Theta(g(n))$ se existirem constantes positivas c_1 e c_2 tais que ela possa ser "encaixadas" entre $c_1 \cdot g(n)$ e $c_2 \cdot g(n)$ para um valor de n suficientemente grande.

2.2 Notação O

Para uma dada função $g(n)$, denotamos por $O(g(n))$ o conjunto de funções

$O(g(n)) = f(n)$: existem duas constantes positivas c e n_0 tais que

$$0 \leq f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0$$

Ou seja, uma função $f(n)$ é igual a $O(g(n))$ caso exista a constante positiva c tal que ela seja maior a $f(n)$ para um n suficientemente grande.

2.3 Notação Ω

Para uma dada função $g(n)$, denotamos por $\Omega(g(n))$ o conjunto de funções

$\Omega(g(n)) = f(n)$: existem duas constantes positivas c e n_0 tais que

$$0 \leq c \cdot g(n) \leq f(n) \text{ para todo } n \geq n_0$$

Ou seja, uma função $f(n)$ é igual a $\Omega(g(n))$ caso exista a constante positiva c tal que ela multiplicado por $g(n)$ seja menor ou igual a $f(n)$ para um n suficientemente grande.

2.4 Notação o

Dada função $g(n)$, a classe $o(g(n))$ é o conjunto de funções $f(n)$ tais que:

$$0 \leq f(n) < c \cdot g(n) \text{ para todo } n \geq n_0$$

2.5 Notação ω

Dada função $g(n)$, a classe $\omega(g(n))$ é o conjunto de funções $f(n)$ tais que:

$$0 \leq c \cdot g(n) < f(n) \text{ para todo } n \geq n_0$$

2.6 Propriedades das notações assintótica

2.6.1 Transitividade

- Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$, então $f(n) = O(h(n))$.
- Se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$, então $f(n) = \Theta(h(n))$.
- Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$, então $f(n) = \Omega(h(n))$.
- Se $f(n) = o(g(n))$ e $g(n) = o(h(n))$, então $f(n) = o(h(n))$.
- Se $f(n) = \omega(g(n))$ e $g(n) = \omega(h(n))$, então $f(n) = \omega(h(n))$.

2.6.2 Reflexividade

- $f(n) = O(f(n))$.
- $f(n) = \Theta(f(n))$.
- $f(n) = \Omega(f(n))$.

2.6.3 Simetria

- $f(n) = \Theta(g(n))$ se, e somente se, $g(n) = \Theta(f(n))$.

2.6.4 Simetria Transposta

- $f(n) = O(g(n))$ se, e somente se, $g(n) = \Omega(f(n))$.
- $f(n) = o(g(n))$ se, e somente se, $g(n) = \omega(f(n))$.

2.7 Complexidade de Tempo

Na análise de algoritmos, é comum expressar a complexidade de tempo em termos de notações assintóticas, que descrevem como o tempo de execução de um algoritmo aumenta com o tamanho da entrada. Aqui estão algumas das notações mais comuns e seus significados:

- $O(1)$: Tempo constante. Isso significa que o tempo de execução não depende do tamanho da entrada; as operações são executadas um número fixo de vezes. Exemplo: acesso a um elemento em uma matriz.
- $O(\log n)$: Tempo logarítmico. Esta notação é usada em algoritmos que dividem repetidamente o problema pela metade, como a busca binária. O tempo aumenta de forma muito lenta à medida que o tamanho da entrada cresce.
- $O(n)$: Tempo linear. O tempo de execução cresce linearmente com o tamanho da entrada. Exemplos incluem a busca linear e a soma de todos os elementos de uma lista.
- $O(n \log n)$: Tempo log-linear. É comum em algoritmos de ordenação eficientes, como o merge sort e o heapsort. O tempo aumenta um pouco mais rápido do que linearmente à medida que o tamanho da entrada cresce.

- $O(n^2)$: Tempo quadrático. Isso ocorre frequentemente quando os dados são processados em pares, com laços de repetição aninhados. Exemplos incluem algoritmos de ordenação ingênuos, como bubble sort e selection sort.
- $O(n^3)$: Tempo cúbico. O tempo de execução cresce cúbicamente com o tamanho da entrada. Um exemplo é a multiplicação de matrizes $n \times n$.
- $O(c^n)$: Complexidade exponencial. Típico de algoritmos que realizam busca exaustiva para resolver um problema. Não são práticos para entradas grandes devido ao crescimento exponencial.
- $O(n!)$: Complexidade fatorial. Pior do que exponencial. Não são úteis do ponto de vista prático, pois crescem muito rapidamente com o tamanho da entrada.

2.8 Por quê calcular limites superiores e inferiores?

Por que calcular limites superiores e inferiores é importante? Esses limites fornecem uma compreensão abrangente do comportamento assintótico de uma função ou algoritmo. Enquanto os limites superiores (como o big O) indicam um limite superior para o crescimento da função, os limites inferiores (como o big Ômega) oferecem uma garantia mínima sobre o crescimento. Juntos, esses limites nos permitem entender o comportamento geral de uma função em diferentes cenários de entrada, ajudando na análise de complexidade de algoritmos e na previsão de seu desempenho em diferentes situações.

Além disso, ao analisar o comportamento de uma função conforme n tende ao infinito, podemos obter insights valiosos sobre sua eficiência em lidar com conjuntos de dados cada vez maiores. Por exemplo, se uma função apresentar um limite superior $O(n^2)$ enquanto n tende ao infinito, isso indica que seu tempo de execução cresce quadraticamente com o tamanho da entrada. Por outro lado, se uma função exibir um limite superior $O(n)$, significa que seu tempo de execução aumenta linearmente com o tamanho da entrada. Compreender como o desempenho de uma função evolui à medida que n cresce é fundamental para a seleção e otimização de algoritmos em problemas de grande escala, onde a eficiência é crucial para lidar com volumes significativos de dados de maneira eficaz e econômica.